Microsoft Foundry

A practical guide to models, agents, tools, and evaluation

In brief

Microsoft Foundry is a platform for building AI applications with models and agents. It brings development and management capabilities together, including model discovery, agent tools, evaluation, and observability. Exact capabilities depend on the service, region, and feature status.

What Foundry brings together

- Models: discover and compare models for a task, then deploy a model endpoint when your application needs one. The catalog spans Microsoft and partner offerings; availability and terms vary by model. [1][2]
- Agents: combine a model with instructions and tools so an application can respond to requests and take actions. Foundry Agent Service supports managed prompt agents and hosted agents, among other ways to build. [1][3]
- Projects: organize work in a Foundry project, where people can build, configure, and manage AI resources with shared project context. [1]
- Operations: use evaluation and observability capabilities to assess quality and inspect behavior across development and production workflows. [1][4]

A useful mental model

A model generates or interprets content. An agent adds instructions, context, and tools around a model. A project provides a place to configure and manage that work. Evaluation and observability help teams understand whether the solution behaves as intended.

Important note

Foundry is a platform, not a guarantee that every model, feature, or deployment option is available in every location or project. Check current model cards, service documentation, and regional availability before making design decisions.

From model to agent

A practical sequence for shaping an AI application

1. Start with the task

Describe the user need, inputs, expected output, and risks. Decide whether a direct model call is sufficient or whether the application needs multi-step reasoning, persistent tools, or managed agent behavior.

2. Select and test a model

Use the Foundry Models catalog to explore models and compare their documented capabilities. Test candidates with representative prompts and data. Consider response quality, latency, cost, safety requirements, and deployment availability; a catalog listing alone does not establish fit for your workload. [2]

3. Choose how to build the agent

- Prompt agent: define instructions, select a model, and connect tools while Foundry manages the agent runtime.
- Hosted agent: package application code when you need greater control over frameworks or runtime behavior.
- Responses API: call the API from an application you operate elsewhere when that architecture is a better fit. [3]

4. Add only the tools you need

Tools extend what an agent can do. Depending on the workflow, tools can include file search, web search, code execution, MCP-connected tools, or custom functions. Give each tool a clear purpose and define the data and actions it is allowed to use. [3]

Design checkpoint

A useful agent is more than a prompt. Specify its task, trusted knowledge, allowed actions, failure behavior, and a way to test its answers before people rely on it.

Keep deployment context in view

Model access, quota, pricing, regional availability, identity, networking, and preview status can affect a design. Verify current project settings and official service guidance before deploying.

Tools and trusted knowledge

How an agent can use information beyond its prompt

Tools connect an agent to capabilities

A tool gives an agent a defined way to retrieve information or request an action. Foundry Agent Service documents built-in options such as file search, web search, and code interpreter, along with MCP servers and custom functions. Tool availability and setup vary by agent type and configuration. [3]

Retrieval-augmented generation (RAG)

RAG combines retrieval with generation. An application searches a connected knowledge source for relevant passages, then supplies retrieved context to a model to help answer a question. In a file-search workflow, source documents inform the answer; the model still generates the response.

- Prepare relevant, current documents and confirm that users are allowed to access them.
- Ask questions that have answers in the source, then verify the response against the retrieved material.
- When evidence is missing or ambiguous, the application should say so rather than presenting a guess as a fact.
- Treat retrieval as grounding assistance, not a guarantee of correctness. Test retrieval and answer quality with realistic questions.

A safe way to think about answers

A response can be fluent and still be unsupported. Check whether its claims match the source, whether important context was omitted, and whether the answer clearly distinguishes documented facts from uncertainty.

Try it

Ask a file-search agent a factual question that the connected document answers, then ask a question that the document does not cover. Compare the answer with the source and check whether uncertainty is handled clearly.

Access and tool boundaries

Use the least access needed for the task. Review identity, permissions, network exposure, and tool side effects. Do not connect sensitive data or allow consequential actions without appropriate authorization, testing, and human oversight.

Evaluation and observability

Test behavior before release and learn from real usage

Evaluate throughout the lifecycle

- Model selection: compare candidate models against representative tasks and quality or safety criteria.
- Application development: build a repeatable test set and evaluate the full workflow, including retrieval, instructions, and tools.
- Production: monitor behavior and investigate issues using available traces, metrics, and evaluation workflows. Microsoft describes evaluation and observability as lifecycle capabilities; exact features vary. [1][4]

What to measure

Choose measures that reflect your use case. Examples include whether an answer is grounded in evidence, whether it addresses the request, whether it follows safety rules, and whether tool calls are appropriate. Use representative examples and human review for high-impact decisions. A single score cannot establish that an agent is safe or correct.

Observability helps explain behavior

Tracing can help teams inspect steps in an AI workflow, while monitoring and evaluation help surface quality or operational concerns. Use available telemetry responsibly: protect sensitive data, limit access, and align retention with organizational requirements. [4]

Demo questions

- What is Microsoft Foundry, and which capabilities does it bring together?
- What is the difference between a prompt agent and a hosted agent?
- How can tools extend an agent beyond a basic model call?
- What does RAG mean, and why should a retrieved answer be checked against its source?
- Why evaluate an AI application before and after deployment?

Sources and further reading

Official Microsoft Learn pages used for this demo brief

The summary in this handout paraphrases public Microsoft documentation. Follow these links for current details, prerequisites, feature status, and implementation guidance.

[1] What is Microsoft Foundry?
https://learn.microsoft.com/en-us/azure/foundry/what-is-foundry
[2] Microsoft Foundry Models overview
https://learn.microsoft.com/en-us/azure/foundry/concepts/foundry-models-overview
[3] What is Microsoft Foundry Agent Service?
https://learn.microsoft.com/en-us/azure/foundry/agents/overview
[4] Observability in Generative AI
https://learn.microsoft.com/en-us/azure/foundry/concepts/observability

Currency

Sources checked October 18, 2026. Product capabilities and documentation can change. Always consult the linked pages before making implementation or purchasing decisions.

Glossary

- Agent: an application that uses a model with instructions and, often, tools to complete a task.
- RAG: retrieval-augmented generation; retrieving relevant context and providing it to a model as it generates a response.
- Observability: visibility into application behavior that can help teams monitor, evaluate, and troubleshoot an AI workflow.